Top 20 Interview Questions — Full C++ Answers

Each one: idea in one line, then complete working code.

ARRAY PATTERNS

1. Maximum Subarray Sum (Kadane's)

Idea: keep a running sum, reset to 0 whenever it goes negative, track the max seen.

```
int maxSubArray(vector<int>& nums) {
    int maxSum = nums[0], curSum = 0;
    for (int n : nums) {
        curSum = max(curSum + n, n);   // reset if curSum+n is worse than n alone
        maxSum = max(maxSum, curSum);
    }
    return maxSum;
}
```

2. Move Zeroes to End

Idea: writePtr tracks next non-zero slot; scan and swap non-zero elements forward.

```
void moveZeroes(vector<int>& nums) {
    int writePtr = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (nums[i] != 0) {
            swap(nums[writePtr], nums[i]);
            writePtr++;
        }
    }
}
```

3. Two Sum

Idea: for each number, check if (target - number) was already seen, store in hashmap.

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> seen;   // value -> index
    for (int i = 0; i < nums.size(); i++) {
        int need = target - nums[i];
        if (seen.count(need)) return {seen[need], i};
        seen[nums[i]] = i;
    }
    return {};
}
```

4. Best Time to Buy & Sell Stock

Idea: track the lowest price seen so far, compute profit at every step, keep the max.

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for (int p : prices) {
        minPrice = min(minPrice, p);
        maxProfit = max(maxProfit, p - minPrice);
    }
    return maxProfit;
}
```

5. Product of Array Except Self

Idea: prefix[i] = product of everything left of i. suffix[i] = product of everything right. Multiply both, no division.

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, 1);
    int prefix = 1;
    for (int i = 0; i < n; i++) { res[i] = prefix; prefix *= nums[i]; }
    int suffix = 1;
    for (int i = n-1; i >= 0; i--) { res[i] *= suffix; suffix *= nums[i]; }
    return res;
```

```
}
```

STACK PATTERNS

6. Valid Parentheses

Idea: push opening brackets, on closing bracket check top of stack matches, else invalid.

```
bool isValid(string s) {
    stack<char> st;
    unordered_map<char,char> pair = {{')','('},{']','['},{'}','{'}};
    for (char c : s) {
        if (c=='('||c=='['||c=='{') st.push(c);
        else {
            if (st.empty() || st.top() != pair[c]) return false;
            st.pop();
        }
    }
    return st.empty();
}
```

7. Sort a Stack

Idea: pop from main stack, insert into a sorted helper stack one at a time (insertion-sort style using recursion or a second stack).

```
void sortedInsert(stack<int>& st, int val) {
    if (st.empty() || st.top() <= val) { st.push(val); return; }
    int top = st.top(); st.pop();
    sortedInsert(st, val);
    st.push(top);
}
void sortStack(stack<int>& st) {
    if (st.empty()) return;
    int top = st.top(); st.pop();
    sortStack(st);
    sortedInsert(st, top);
}
```

8. Next Greater Element

Idea: monotonic decreasing stack — pop smaller elements when a bigger one arrives, that bigger one is their answer.

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st;   // stores indices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.top()] < nums[i]) {
            res[st.top()] = nums[i];
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

9. Min Stack

Idea: keep a second stack that always tracks the current minimum alongside the main stack.

```
class MinStack {
    stack<int> main, minSt;
public:
    void push(int val) {
        main.push(val);
        if (minSt.empty() || val <= minSt.top()) minSt.push(val);
        else minSt.push(minSt.top());
    }
    void pop() { main.pop(); minSt.pop(); }
    int top() { return main.top(); }
    int getMin() { return minSt.top(); }
};
```

LINKED LIST PATTERNS

10. Reverse Linked List

```
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *curr = head;
    while (curr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

11. Detect Cycle (Floyd's)

Idea: slow moves 1 step, fast moves 2 steps. If they meet, there's a cycle.

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

12. Find Middle Node

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;   // when fast hits end, slow is at middle
}
```

13. Merge Two Sorted Lists

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

14. Nth Node from End

Idea: move fast pointer n steps ahead first, then move both together — gap stays n.

```
ListNode* nthFromEnd(ListNode* head, int n) {
    ListNode *fast = head, *slow = head;
    for (int i = 0; i < n; i++) fast = fast->next;
    if (!fast) return head;   // n equals list length
    while (fast->next) { fast = fast->next; slow = slow->next; }
    return slow->next;
}
```

TREE / DFS PATTERNS

15. Maximum Path Sum

```
int ans = INT_MIN;
int dfs(TreeNode* root) {
    if (!root) return 0;
    int left = max(0, dfs(root->left));
    int right = max(0, dfs(root->right));
    ans = max(ans, root->val + left + right);
    return root->val + max(left, right);
}
int maxPathSum(TreeNode* root) { dfs(root); return ans; }
```

16. Level Order Traversal (BFS)

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
```

17. Height of Binary Tree

```
int height(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}
```

18. Diameter of Tree

Idea: diameter through a node = left height + right height. Track global max while computing height.

```
int diameter = 0;
int heightDia(TreeNode* root) {
    if (!root) return 0;
    int left = heightDia(root->left);
    int right = heightDia(root->right);
    diameter = max(diameter, left + right);
    return 1 + max(left, right);
}
```

GRAPH PATTERNS

19. Number of Islands

```
void dfs(vector<vector<char>>& grid, int r, int c) {
    if (r<0 || c<0 || r>=grid.size() || c>=grid[0].size() || grid[r][c]=='0') return;
    grid[r][c] = '0';
    dfs(grid,r+1,c); dfs(grid,r-1,c); dfs(grid,r,c+1); dfs(grid,r,c-1);
}
int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for (int i = 0; i < grid.size(); i++)
        for (int j = 0; j < grid[0].size(); j++)
            if (grid[i][j] == '1') { count++; dfs(grid, i, j); }
    return count;
}
```

HEAP / STREAM PATTERNS

20. Median from Data Stream

Idea: maxHeap holds the smaller half, minHeap holds the larger half. Median sits at the boundary.

```
priority_queue<int> maxHeap;                              // smaller half
priority_queue<int, vector<int>, greater<int>> minHeap; // larger half

void addNum(int num) {
    maxHeap.push(num);
    minHeap.push(maxHeap.top()); maxHeap.pop();
    if (minHeap.size() > maxHeap.size()) {
        maxHeap.push(minHeap.top()); minHeap.pop();
    }
}
double findMedian() {
    if (maxHeap.size() > minHeap.size()) return maxHeap.top();
    return (maxHeap.top() + minHeap.top()) / 2.0;
}
```

Pattern Memory Table

```
max/min subarray         -> Kadane
stream median             -> Two Heaps
grid / island              -> DFS / BFS
sorted array search        -> Binary Search
top K elements              -> Heap
parentheses / stack stuff   -> Stack
pairs / sorted scan          -> Two pointers
linked list movement          -> Slow / Fast pointers
```